Zombie Hamlet Report

CMPT 276 D100
Group 8

Niki saberi -301608011

Ajay Partap Sekhon - 301600629

Amitoj Kochar - 301591083

Manya Asri - 301579204

Introduction

We built Zombie Hamlet using a modular design that makes the game easy to expand in the future. Our goal was to create a framework where new elements like traps, food items, and enemies could be added without rewriting existing code. This approach also helped our team work efficiently, as different members could develop separate components without conflicts.

General Implementation

The game runs on a central game loop in the GamePanel class that updates positions, checks collisions, and maintains a consistent frame rate. An InputHandler class processes keyboard controls in real time, determining how the player moves and acts.

The game world is built through a layered rendering system. The TileManager reads map data from text files and displays the environment using tile sprites, creating walls and walkable areas without hardcoding positions.

All game objects follow an inheritance structure. MovingEntity and StillEntity serve as base classes for shared properties like position and health. Specific entities like Zombie, Player, Trap, and Food extend these classes with their own unique behaviors.

Collision detection uses a straightforward event system - when the player reaches the same position as an interactable object, collision methods trigger automatically. This system is easy to extend for future additions.

Each class has one clear responsibility, making the code organized and easy to debug. Well-defined interfaces allow different modules to communicate effectively while maintaining separation between components.

2. Changes Made from Initial Design

We made several major improvements to our original game design. The biggest change was to the zombie movement system. At first, zombies moved in straight lines toward the player, but they kept getting stuck on walls. We fixed this by creating a new pathfinding system that uses the BFS algorithm. This system includes new Pathfinder and Node classes that help zombies find their way around obstacles and through mazes.

We also completely changed how we build our game maps. Our first idea was to create a Wall class for each barrier, but this took too much time and wasn't flexible. We then tried making a colored maze where zombies would follow color paths, but this caused more problems with zombies getting stuck and sizing issues. Finally, we created a much better system using text files where numbers represent different tiles. Our TileManager class reads these files and builds the game world, making it easy to create and change levels.

We also upgraded our graphics from simple still images to animated characters that move more naturally. We added a difficulty system that changes how many zombies appear, how fast they

move, and how many resources are available at each challenge level such as coins and zombie speed

3. Roles and Responsibilities:

Amitoj —Traps, Player(HealthBar), Coin, StillEntity, Gamepanel, Report

Created Trap and Coin classes extending StillEntity. The Trap class implements a damaging trap upon player position overlap and an armed trap mechanism (isArmed, cooldown, trigger()). The overlap mechanism implements animated sprites for an animation for armed and not armed. The Coin class implements a six-frame sprites as well to animate a rotating coin. Coins also implement a collect(Player) mechanism to increase the coins variable from the player object.I helped integrate HealthBar on the player side to visually represent health, by the hearts changing from a full heart image to an empty heart image every time the player takes damage. Also helped GamePanel implementations for coins and traps to properly function within the game.

Ajay — Implemented the TileManager, KeyHandler, EscapeGate, Position, with major contributions to Player and the Gamepanel. Also designed the animations for the characters and some props, while sourcing others. Also worked on the SoundManager class and made simple sound tracks and effects for the game. Worked on the winning and game over screens while also implementing the ENTER to retry or ESC to go to the main menu mechanic. Major contribution to the gameplay (player movement, animation, map design).

Manya — Worked on multiple project components, including Zombie, UI, Difficulty, PathFinder, Node, MovingEntity, and GamePanel classes. Implemented entity movement, pathfinding, and UI updates, along with gameplay and usability improvements such as food respawn every 10 seconds. Added JavaDoc comments for proper documentation. Designed and implemented start and level buttons with full page layout, integrating them into the KeyHandler class for actions like pausing and quitting the game.

Niki — StillEntity, Food, Art, Gamepanel, Report,Player, Zombie
Implemented the initial color based pathfinding. Created the game's secondary characters and backgrounds. Created the StillEntity class as a base for stationary objects and the Food class for collectible health items. Modified the Player class to handle movement and health, stop at walls and gain health points from the Food class. Modified the Zombie class to use a pathfinding algorithm to chase players and turn when stuck. Changed the Game Panel according to all these modifications.

4. The libraries we used and why:

1. Window & UI (Swing)

- javax.swing.JFrame: The main window that holds all UI components.
- javax.swing.JPanel: The "canvas" we extend to create GamePanel and MenuPane.
- java.awt.Dimension: Used to set the preferred size of the window and panels.

2. Graphics & Drawing (AWT)

- java.awt.Graphics: The basic drawing tool, provided by the system.
- java.awt.Graphics2D: The advanced drawing tool, used to draw sprites (BufferedImage) and shapes.
- java.awt.image.BufferedImage: Used to store loaded sprite images, like the player, zombies, and tiles.
- javax.imageio.ImageIO: Used to read image files (e.g., .png, .jpg) from the resource folder.
- java.awt.Color: Used to set colors for backgrounds, text, and UI elements.
- java.awt.Font: Used to load and define custom fonts from files (like .otf or .ttf).
- java.awt.FontMetrics: Used to measure text to find its width/height for centering.

3. Sound (Sampled)

- javax.sound.sampled.AudioSystem: Gets the audio files from the system.
- javax.sound.sampled.Clip: Represents a loaded audio file (e.g., .wav) that can be played, looped, or stopped.
- javax.sound.sampled.FloatControl: Used to change the volume (gain) of a Clip.

4. Event Handling (User Input)

- java.awt.event.KeyListener: The interface that listens for key presses.
- java.awt.event.KeyEvent: The object that describes which key was pressed (e.g., VK_ENTER, VK_W).

5. Data Structures & Utilities

- java.util.ArrayList: Used to hold the lists of zombies, coins, traps, and food.
- java.util.Random: Used for the random placement of coins and food.
- java.util.PriorityQueue: The core data structure for the A* pathfinding "open list."
- java.util.List: The interface for our ArrayLists and the pathfinding path list.

6. File I/O & Exceptions

- java.io.InputStream: Used to read data from a resource file (like a font, map, or image).
- java.io.IOException: An error that must be caught when reading files.
- java.net.URL: Used to find the location of a resource file, like a .wav file.

5.Measures Taken to Enhance Code Quality

We focused on writing clean, organized code that would be easy to understand and maintain. We used modular design which means each class has one clear job. The Pathfinder class only handles zombie pathfinding, the TileManager only deals with maps, and the Player class only manages player actions. This separation made our code much easier to work with!

We kept our code well-structured with consistent naming and logical organization. We used encapsulation by making variables private and using methods to access them, which kept our data safe and prevented errors.

We also made sure classes were independent from each other, so changing one part of the game wouldn't break other parts. This modular approach makes our code flexible for future improvements and easier to fix when problems arise.

6. Difficulties We Faced

The hardest problem was making the zombies move properly through the maze. Our first pathfinding system didn't work well, and zombies kept getting stuck. It took many tries to get the BFS algorithm working correctly with priority queues.

Changing our map system also caused a lot of problems. The colored maze approach created issues with zombie navigation and collision detection. When we switched to the text file system, we had to make sure it worked with both the zombie pathfinding and wall collisions.

Keeping the game screen size consistent was another major issue which took a while to get fixed. The start page of the game was bigger than the actual game so when in the playing state the game would render but have a huge black gap on the sides.

We also had trouble loading game images consistently due to some directory structural issues. As we added character animations, we faced challenges with timing the animations properly.

Getting the difficulty balance right was another challenge. We had to test repeatedly to find the right number of zombies, their speeds, and resource amounts for each difficulty level to make the game challenging but not frustrating.

The user interface implementation was also difficult, especially when designing the menu system. We faced challenges with button placement and functionality, and struggled to connect different game screens through keyboard commands. Making the ESC key navigate back to the main menu and the ENTER key trigger game replays demanded careful integration of input handling with our game management.